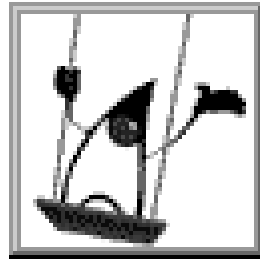


Java Foundation Classes



Swing

JFC and Swing

Java Foundation Classes (JFC)

Swing:

- Swing components
- Pluggable Look and Feel

- Accessibility API
- Java2D API
- Drag and Drop API

JFC

Accessibility API

- allows assistive technologies to be used with Java components
 - e.g. screen readers, screen magnifiers, speech recognition

Java2D API

- more advanced graphics drawing facilities
 - e.g. more sophisticated geometry, transformations, painting styles, image processing

Drag-and-Drop API

- allows dragging and dropping between Java and non-Java applications

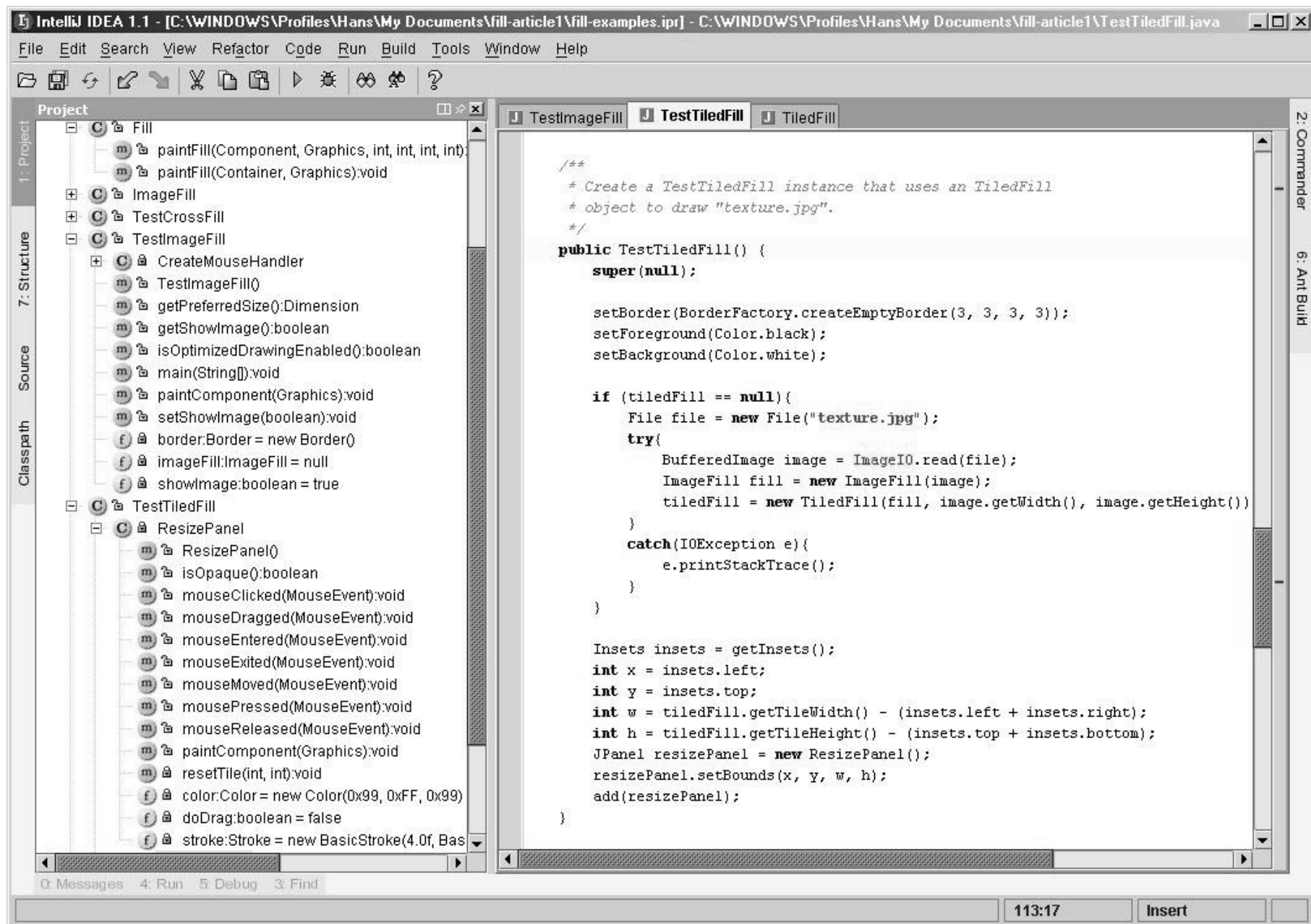
JFC

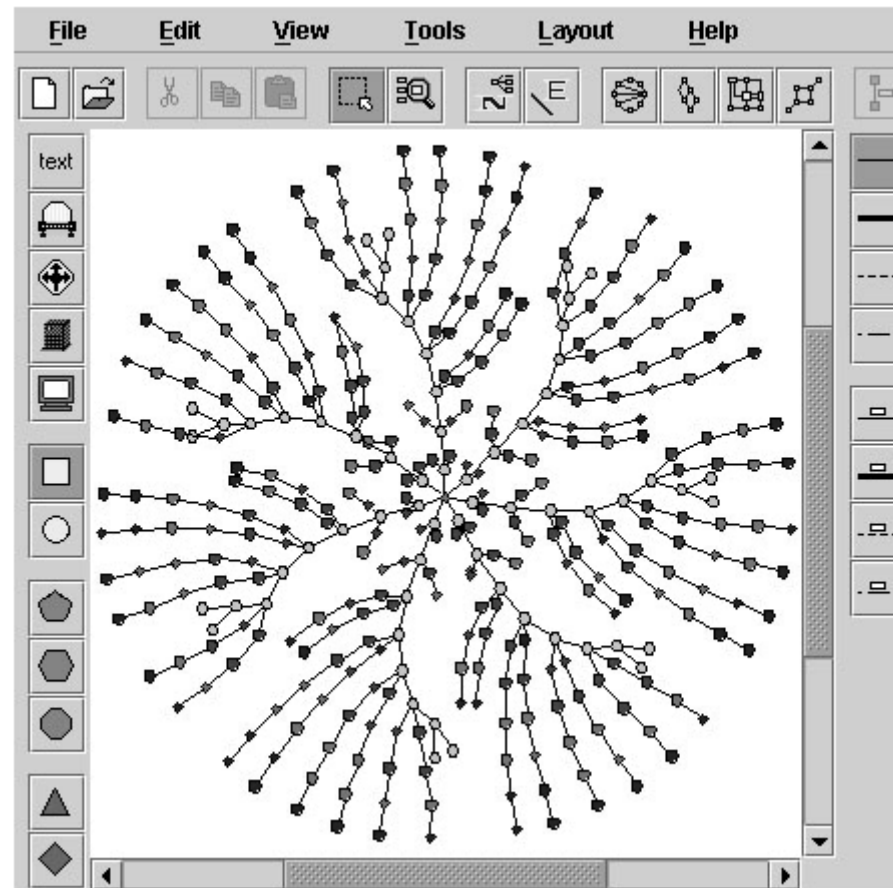
Swing

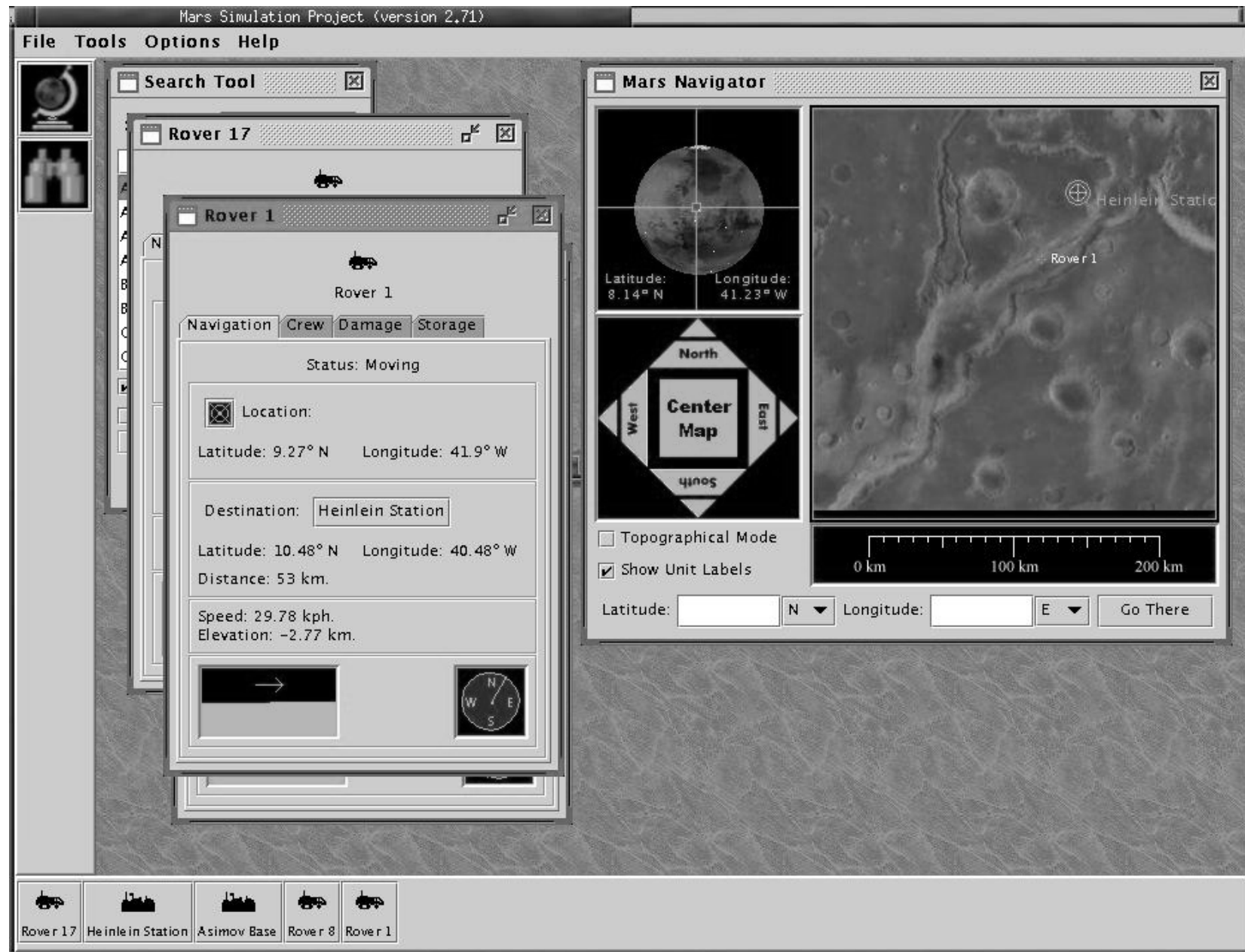
- interface components written in Java
- non-window-system-specific
- customizable

Pluggable Look-and-Feel

- switch the look of a suite of applications without having to re-jig the components of each one
- a number of standard looks and feels provided

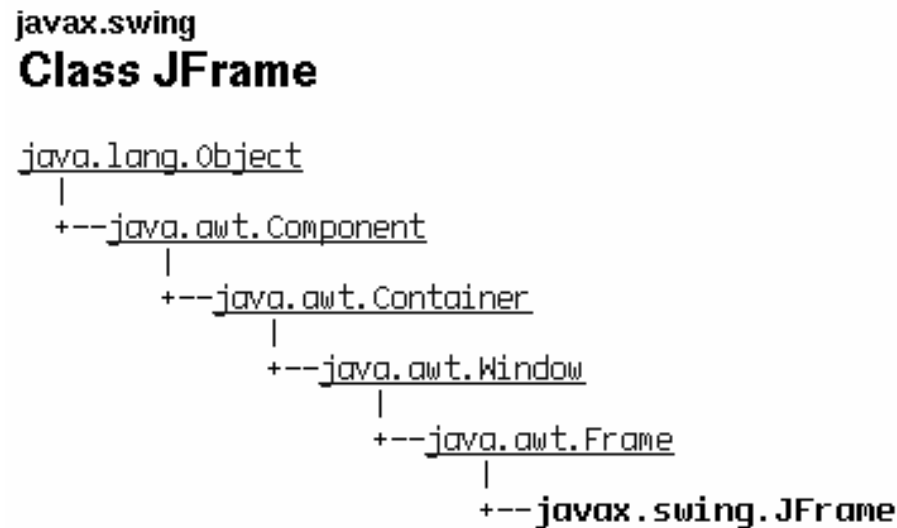






Swing and AWT

- Swing has versions of all the components supplied by the Abstract Windowing Toolkit (AWT)
- ...plus lots more!
- Swing components are typically more specialized versions of (are subclasses of) AWT components, e.g.



Swing Components

Containers

- Top-Level
- General-Purpose
- Special-Purpose

Atomic Components

- Basic Controls
- Information Displays
 - Uneditable
 - Editable

Top-level Containers

At the top of any containment hierarchy, you will find one of these:

Applet

JApplet

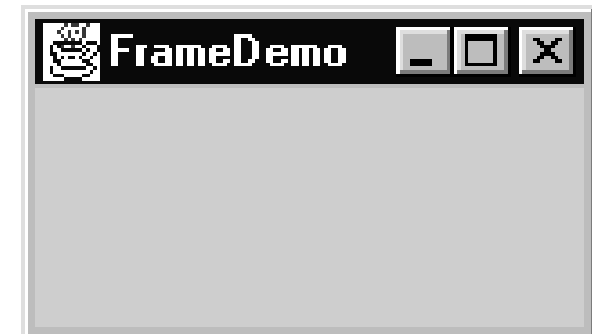
Dialogue



JDialog

JOptionPane

Frame

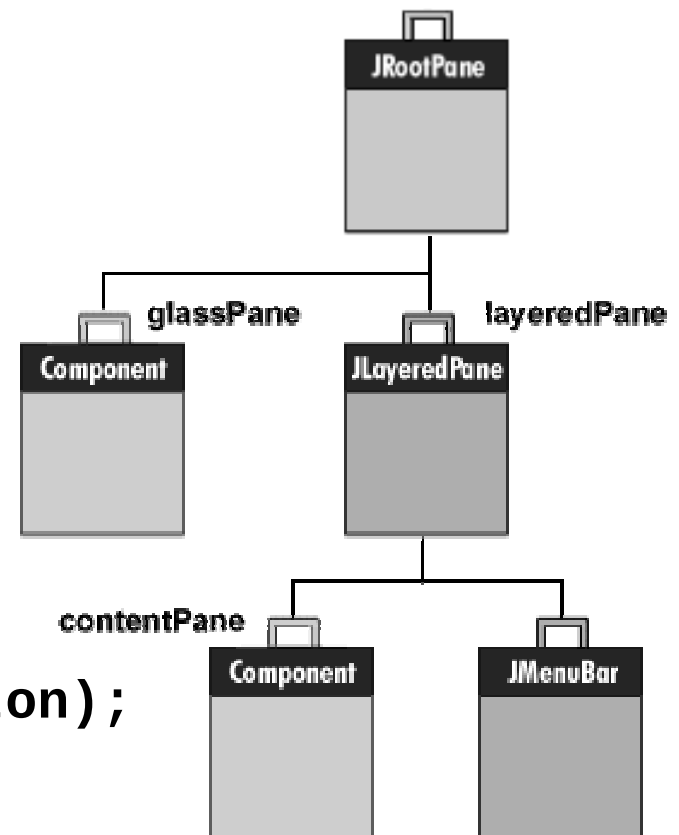
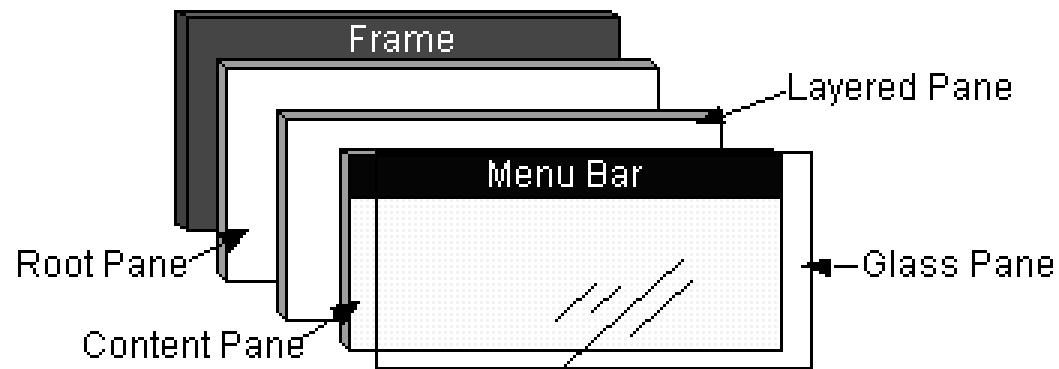


JFrame

Panes

- With Swing, we cannot add directly to a JApplet or JFrame. These top-level containers always have a “contentPane” (a **Container**), and we must do
getContentPane().add(myBigRedButton);
 - (not strictly required in Java 1.5, but is in Java 1.4.*)
- And besides the “contentPane”, Swing has the concepts of “rootPane”, “layeredPane” and “glassPane”, allowing quite sophisticated presentation.

Visualizing the Panes



`getContentPane().add(myBigRedButton);`

General-purpose Containers

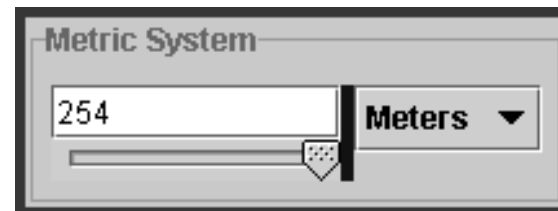
(Intermediate containers of many uses)

Toolbar



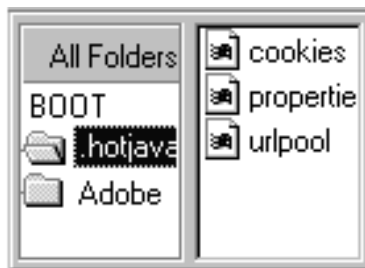
JToolBar

Panel



JPanel

Split Pane



JSplitPane

Scroll Pane



JScrollPane

Tabbed Pane

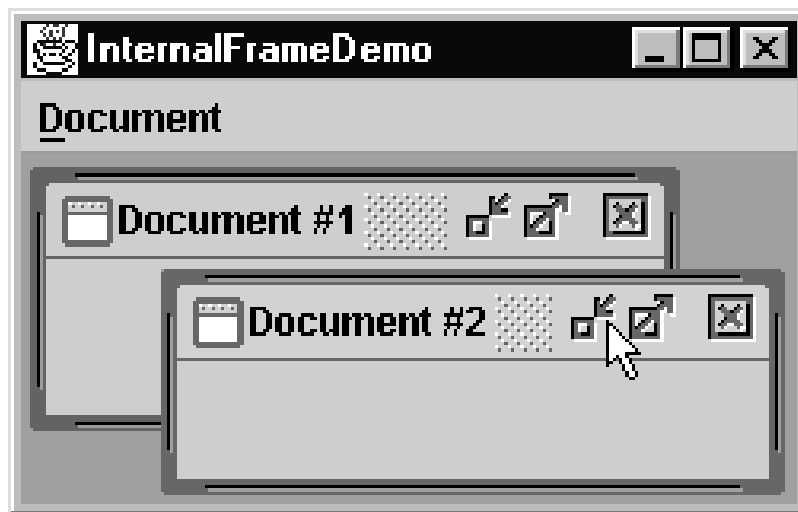


JTabbedPane

Special-purpose Containers

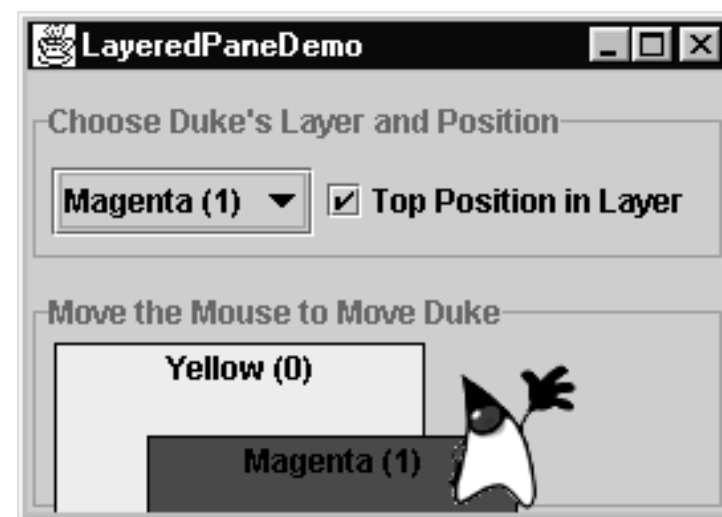
(Intermediate containers with specific roles)

Internal Frame



JInternalFrame

Layered Pane



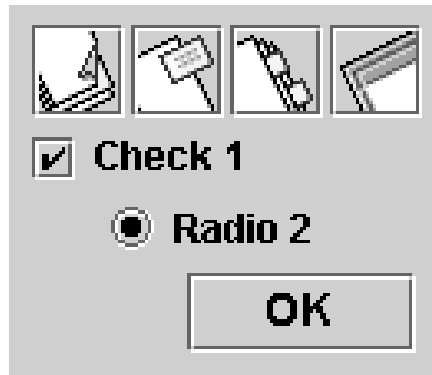
JLayeredPane

Root Pane

JRootPane

Controls

Buttons



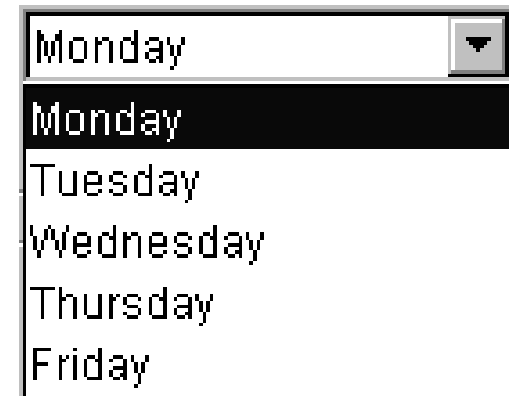
Jbutton

List



JList

Combo Box



JComboBox

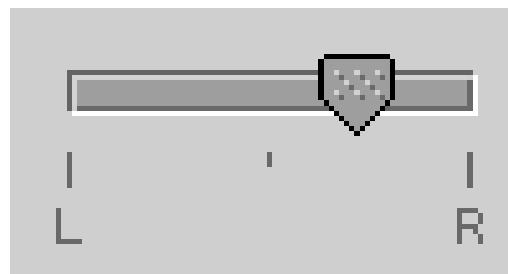
Menu



JMenu

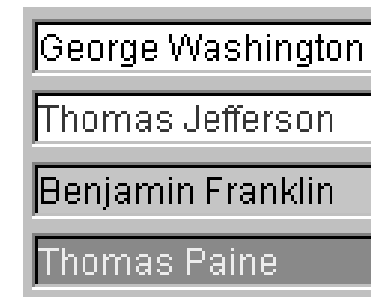
JMenuItem

Slider



JSlider

Text field

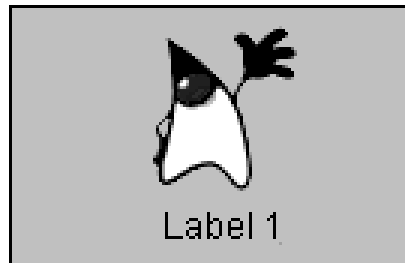


JTextField

Information Displays

(uneditable, solely to give information)

Label



JLabel

Progress Bar



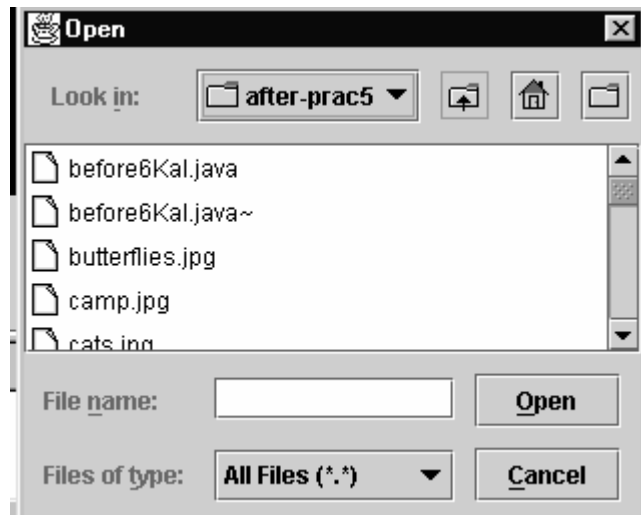
JProgressBar

ToolTip



JToolTip

File Chooser



JFileChooser

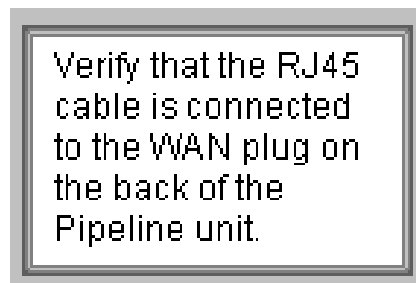
Table

First Na...	Last Name
Mark	Andrews
Tom	Ball
Alan	Chung
Jeff	Dinkins

JTable

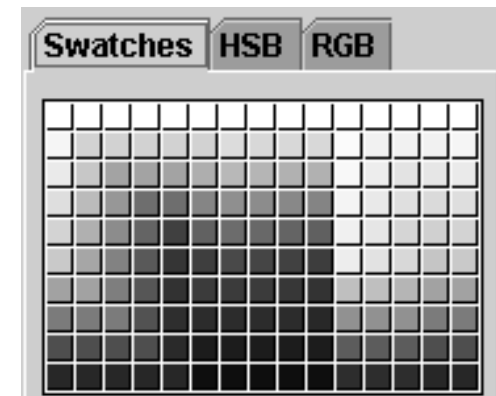
Information Displays

(editable, formatted)



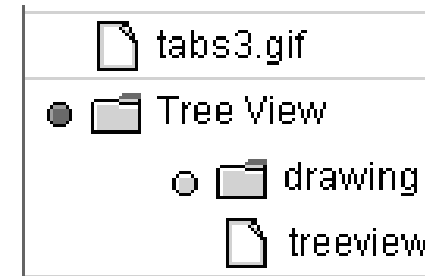
JTextArea

Colour Chooser



JColorChooser

Tree



JTree

Layout Managers

(standard ones)

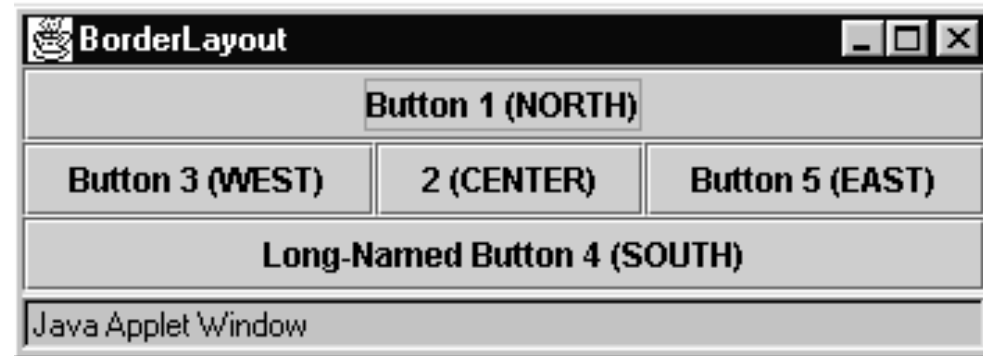
Managers available:

- Flow
- Card
- Border
- Box
- Grid
- GridBag (complicated!)
- customized

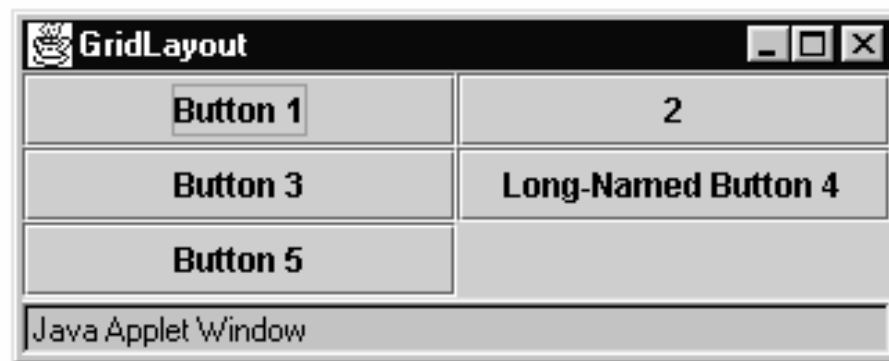
BoxLayout



BorderLayout



GridLayout



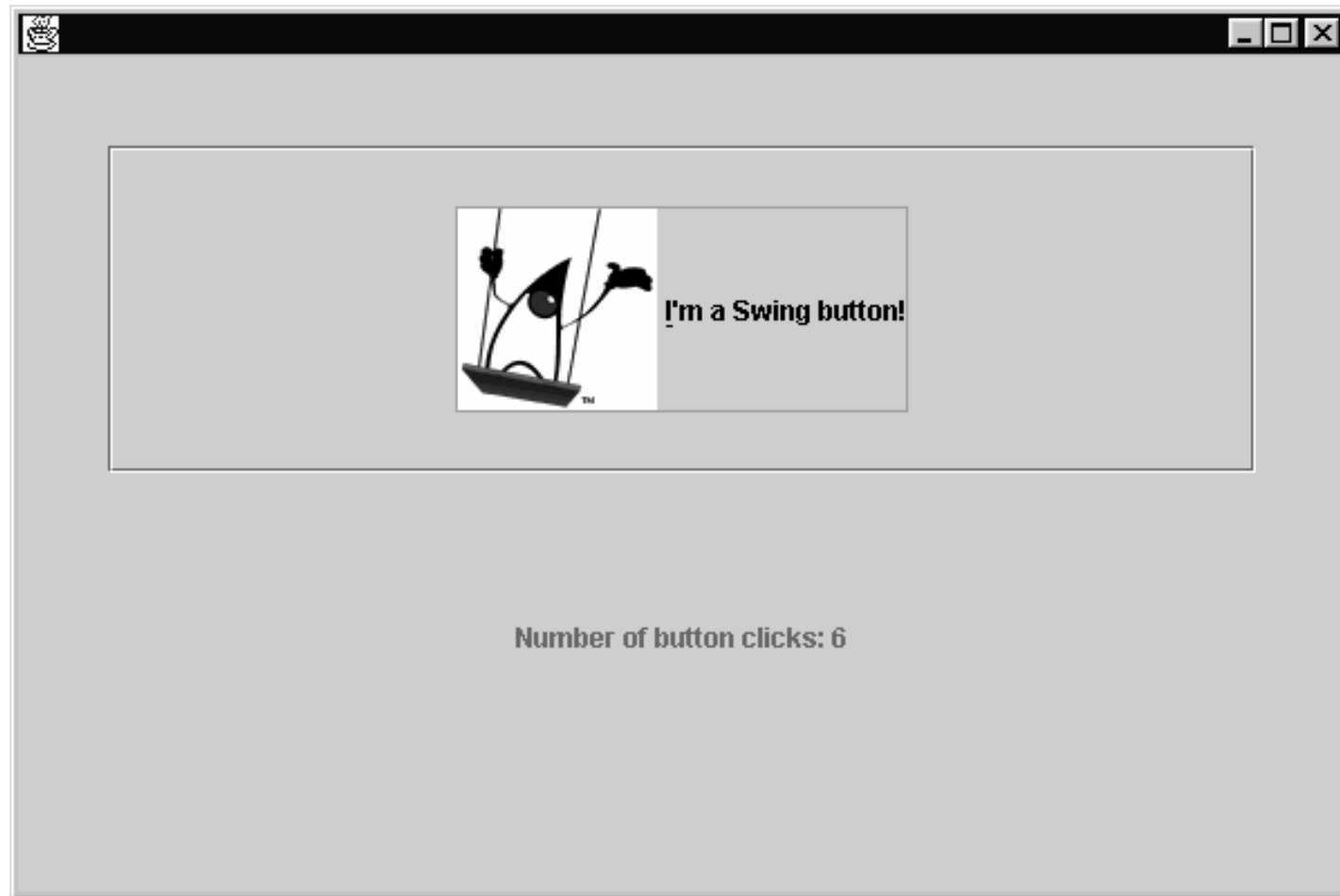
GridBagLayout



Actionable benefits to your business

HyperlinkListener InternalFrameListener
MenuItemListener MouseListener ItemListener
KeyListener MouseInputListener
ListSelectionListener TreeModelListener
ListDataListener MenuKeyListener
MenuDragMouseListener MouseMotionListener
TableColumnModelListener PopupMenuListener
TableModelListener TreeExpansionListener
TreeSelectionListener UndoableEditListener
TreeWillExpandListener WindowListener

Example SwingApplication



```

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class SwingApplication extends JFrame implements ActionListener {

    private String labelPrefix = "Number of button clicks: ";
    JLabel label;
    private int numClicks = 0;

    public SwingApplication() {
        label = new JLabel(labelPrefix + "0", JLabel.CENTER);

        JButton button = new JButton("I'm a Swing button!", new
                                     ImageIcon("duke.gif"));

        button.addActionListener(this);

        JPanel panel = new JPanel();

        // adds a space of 30 pixels around the edge of the panel
        panel.setBorder(BorderFactory.
                        createEmptyBorder(40,40,40,40));
        panel.setLayout(new GridLayout(2,1));
        panel.add(button);
        panel.add(label);

        getContentPane().add(panel, BorderLayout.CENTER);
    }
}

```

```

public void actionPerformed(ActionEvent e) {
    numClicks++;
    label.setText(labelPrefix + numClicks);
}

public static void main(String[] args) {

    // Create the top-level frame
    SwingApplication app = new SwingApplication();

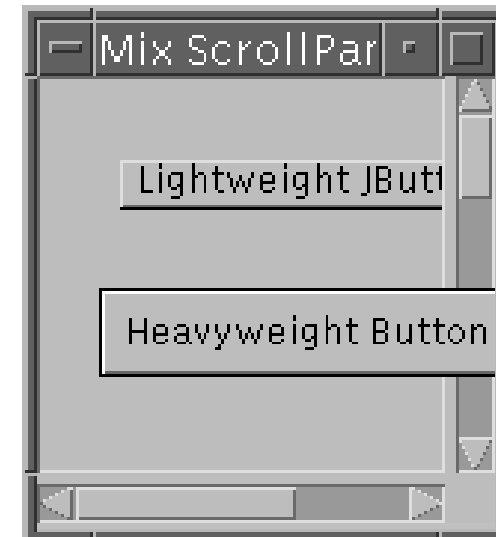
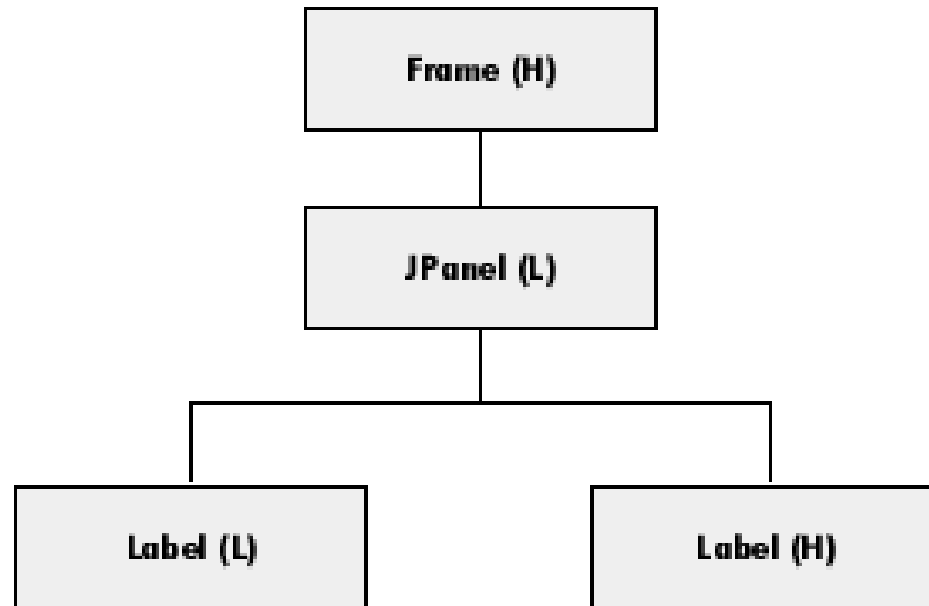
    // Make sure the frame can exit
    app.addWindowListener(new WindowAdapter() {
        public void windowClosing(WindowEvent e) {
            System.exit(0);
        }
    });
    app.setSize(600,400);
    app.setVisible(true);
}
}

```

Heavy and Light Components

- “Heavy” components have their own screen resources, “light” components borrow them
 - **AWT** components are heavy
 - **Swing** components are light
- Mixing them causes problems, because of:
 - lightweights can be transparent, heavies are opaque
 - heavies always appear rectangular
 - mouse events on a lightweight component fall through to the parent of the component - not so with heavies
 - heavy components are always on top

Heavy and Light



More Swing

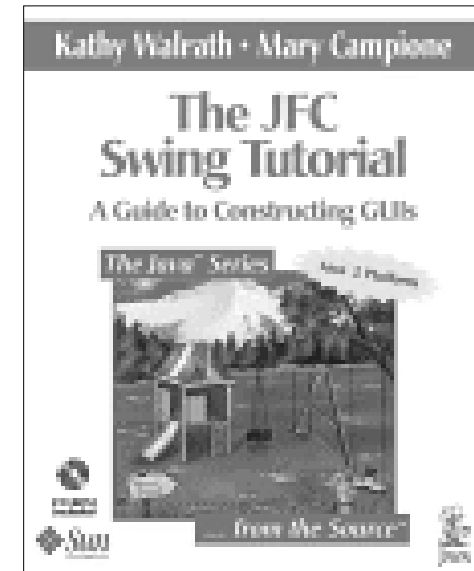
(Information for if you want to go further into Swing)

Many Java books cover some Swing

- One book is by Mary Campione and Kathy Walrath:

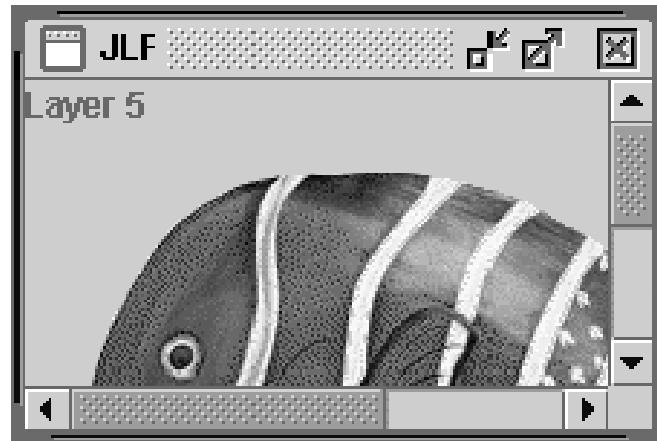
“The JFC Swing tutorial”

- This book is the portable version
- of the online Swing tutorial, at <http://java.sun.com/docs/books/tutorial/uiswing/>
- More information than you could ever possibly want at the sun site, java.sun.com

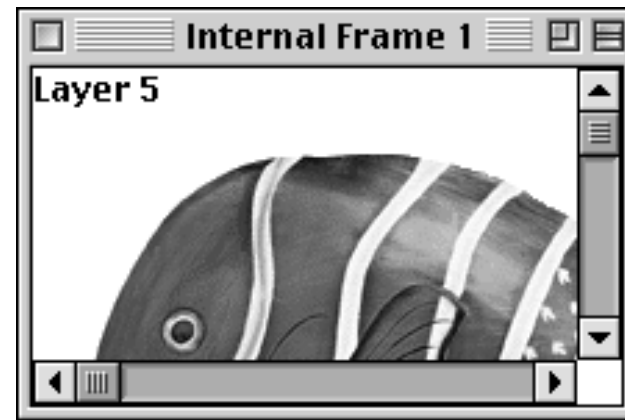


Plug-in Look and Feel Samples

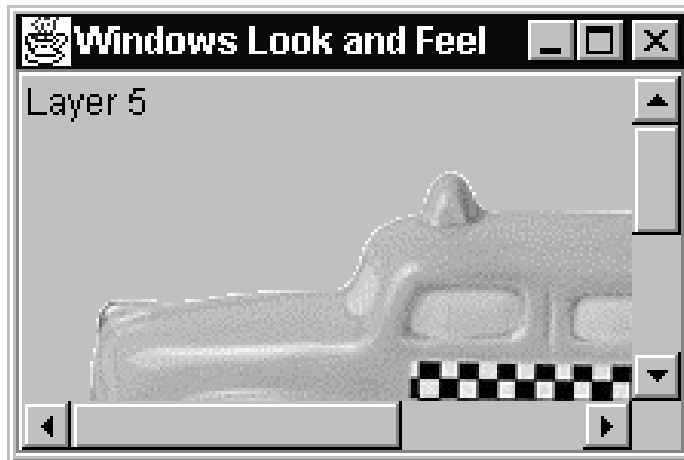
Java



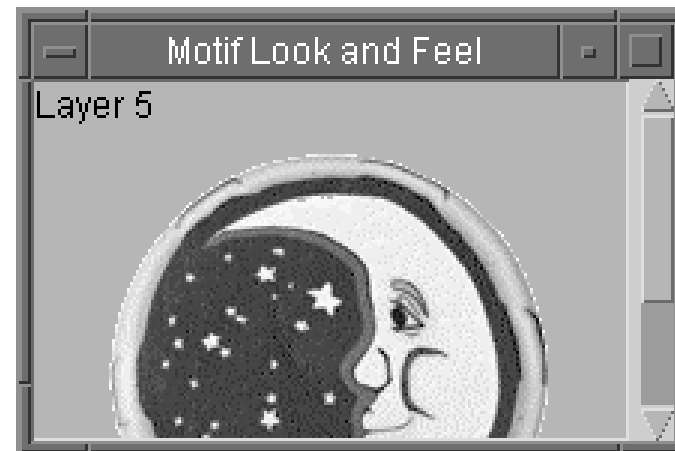
MacOS



Windows



Motif



Java Look and Feel Guidelines

- Very comprehensive, a valuable resource
- Book form, as well as online at
`java.sun.com/products/jlf/`
- Covers structural design, as well as visual/layout guidelines
- Detailed guidelines on various topics, including
 - Interface components
 - Mouse behaviour
 - Using graphics

Look and Feel Graphics

Development



General



Media



Table



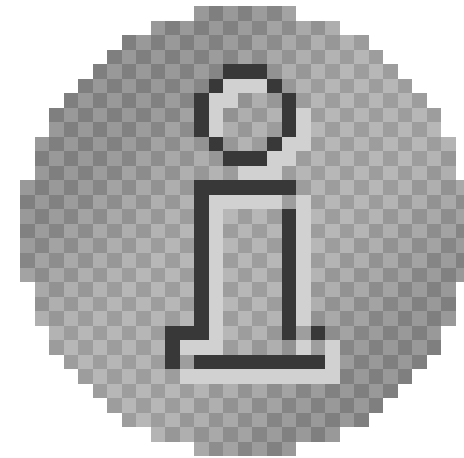
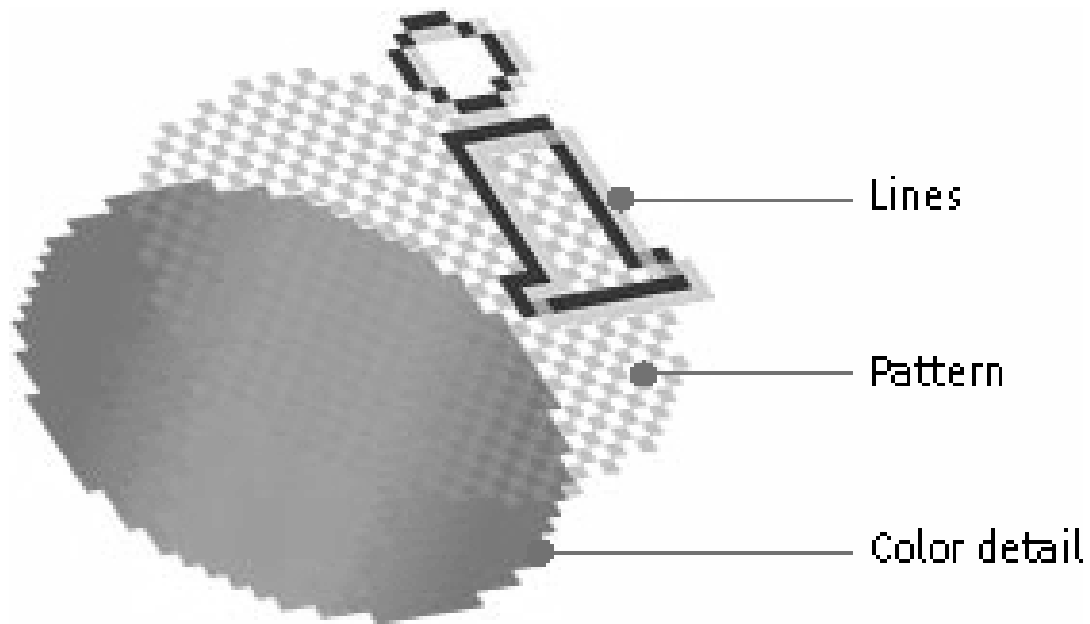
Navigation



Text



Extract from Graphics Guidelines



End Of Lecture

- Next lecture is on Java2D